



Criptografía y seguridad

Grupo 82

Grado en Ingeniería informática

Práctica 1

Héctor Fuertes Muñoz - 100522401 - 100522401@alumnos.uc3m.es

Xinbo Chen Chen - 100495902 - 100495902@alumnos.uc3m.es

https://github.com/100522401/p1_cripto.git

1. Introducción: Objetivos y estructura interna.....	3
1.1. Propósito de la aplicación.....	3
2. Firma digital.....	3
2.1 Aplicaciones de la firma digital.....	3
2.2 Algoritmos usados.....	4
2.2.1 RSA-2048 (Generación de claves).....	4
2.2.2 SHA-256.....	4
2.2.3 RSA-PSS.....	4
2.3 Gestión de claves y firmas.....	5
2.3.1 Gestión de Claves Privadas (RSA).....	5
2.3.2 Gestión de Claves Públicas.....	5
2.3.3 Gestión de Firmas Digitales.....	5
3. Certificados y PKI (Infraestructura de Clave Pública).....	5
3.1 Jerarquía de autoridades.....	5
3.2 Configuración y Justificación.....	6
3.3 Implementación.....	6
3.3.1 Estructura interna de AC1.....	7
3.3.2 Funcionamiento de la AC1.....	7
3.4 Generación de certificados digitales.....	9
4. Pruebas realizadas.....	9
4.1. Registro de usuarios.....	9
4.2. Inicio de sesión.....	10
4.3. Cifrado de archivos.....	10
4.4 Descifrado de archivos.....	10
4.5 Verificación de firmas.....	10
4.6 Verificación de certificados X.509.....	11

1. Introducción: Objetivos y estructura interna.

1.1. Propósito de la aplicación

La aplicación que hemos desarrollado es un **sistema local de gestión y protección de archivos** que permite a los usuarios **registrar sus cuentas, autenticarse de forma segura y cifrar y descifrar archivos** mediante técnicas criptográficas.

El objetivo principal de la aplicación es proporcionar una herramienta que aplique **conceptos criptográficos y de seguridad informática modernos**, permitiéndonos comprender cómo funcionan y cómo se integran estos mecanismos en una aplicación real.

Nuestra aplicación funciona de forma **completamente local**, por lo que toda la **gestión de claves** se realiza en la máquina del usuario.

Para elevar este proyecto al nivel de las interfaces gráficas, hemos hecho uso de **Tkinter**, un framework de Python orientado a la creación de estas interfaces sencillas.

2. Firma digital

2.1 Aplicaciones de la firma digital

La **firma digital** es un mecanismo criptográfico basado en las claves asimétricas que **permite vincular un archivo con la identidad del usuario que lo generó**, garantizando además que el **archivo no fue modificado** desde su creación. A diferencia de la firma manuscrita, la firma digital se calcula haciendo uso de la **clave privada** del usuario y puede verificarse posteriormente con su **clave pública**.

En la aplicación, la firma digital se aplica automáticamente cada vez que un usuario cifra un archivo. Antes de almacenarlo, el sistema genera una firma digital sobre el contenido original del documento usando la clave privada del usuario. La firma digital generada se almacena en el fichero de metadatos del archivo cifrado, lo que permitirá verificar posteriormente su origen e integridad.

```
metadata = {
    "filename": filename,
    "signer": username,
    "enc_key_user": base64.b64encode(enc_key_user).decode(),
    "enc_key_admin": base64.b64encode(enc_key_admin).decode() if enc_key_admin else "",
    "algorithm": "AES-256-GCM",
    "signature": base64.b64encode(signature).decode(),
    "signature_algorithm": get_signature_algorithm_info()
}
```

Metadatos del archivo cifrado

La aplicación utiliza la **firma digital** para garantizar tres principios fundamentales:

- **Autenticación del autor:** permite **identificar que usuario ha generado y cifrado el archivo**, ya que únicamente el propietario de la clave privada es capaz de emitir esa firma.
- **Integridad:** Cualquier modificación en el archivo modifica la firma, de modo que **cualquier manipulación o alteración, se detectaría automáticamente**.
- **No repudio:** la clave pública del usuario está incluida en un certificado X.509 emitido por la Autoridad de Certificación (AC1) lo que garantiza que el archivo firmado está vinculado con el usuario firmante. Es por esto que, si se verifica correctamente con su clave pública certificada, el usuario no podrá negar haber firmado el archivo, ya que solo él posee la privada capaz de emitir dicha firma.

2.2 Algoritmos usados

Para la implementación de la firma digital, hemos seleccionado el siguiente conjunto de algoritmos de la librería *criptography* ya que cumplen con los estándares de seguridad actuales y por tanto, no están obsoletos ni son vulnerables.

2.2.1 RSA-2048 (Generación de claves)

Aunque ya fue mencionado en la entrega anterior, es importante mencionar que las claves pública y privada del usuario son generadas mediante este algoritmo.

En cuanto a la implementación hemos optado por **RSA-2048** un tamaño de clave de **2048 bits**, con un esquema de padding **OAEP** (Optimal Asymmetric Encryption Padding) con la función hash **SHA-256**, garantizando seguridad frente a ataques de *padding* o relleno.

2.2.2 SHA-256

Para generar la firma, se requiere hacer un resumen o hash del contenido a cifrar antes de firmarlo. Esto servirá para detectar posibles modificaciones o alteraciones tras haber firmado, lo que dota al mecanismo de integridad del mensaje.

Hemos optado por **SHA-256** ya que ofrece la resistencia a colisiones necesaria para asegurar que dos archivos diferentes no produzcan la misma firma, lo que también nos garantiza integridad.

2.2.3 RSA-PSS

RSA-PSS (Probabilistic Signature Scheme) es el mecanismo que hemos usado para generar y verificar la firma digital. Para la generación de la firma utiliza la clave privada del usuario.

Este algoritmo aporta al sistema una gran seguridad matemática. Además es un algoritmo **probabilístico**, es decir, si firmamos dos veces el mismo documento las dos firmas generadas serán distintas (aunque ambas serán válidas). Esto dota al sistema de robustez frente a ataques de maleabilidad.

```
signature = private_key.sign(
    plaintext,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)
```

Implementación de la firma. Rojo: RSA-PSS, Verde: SHA-256

2.3 Gestión de claves y firmas

2.3.1 Gestión de Claves Privadas (RSA)

Al registrarse, el sistema le genera al usuario automáticamente un par de claves RSA. La clave privada nunca se almacena en texto plano. Por ello, antes de guardarse en el fichero *users.json*, se cifra con la contraseña.

2.3.2 Gestión de Claves Públicas

Las claves públicas no son secretas, por lo que se almacenan codificadas en Base64 en el fichero *users.json*. Es importante mencionar que, el path del certificado que valida la clave pública del usuario y, en su defecto, el path de la Solicitud de Firma del Certificado (CSR) también se almacenan junto a la clave pública en *users.json*.

```
"user1": {
  "salt": "vYtFjSFazqUSdPC5TAqGpQ==",
  "password_hash": "skHtnvzhDzD3KDQ9s92rzb09i+3F1",
  "public_key": "LS0tLS1CRUdJTiBQVU7MSUMgS0VZLS0t",
  "private_key_enc": "LS0tLS1CRUdJTiBFTkNSWVBURUQ",
  "rol": "user",
  "cert_path": "AC1/nuevoscerts/user1.crt.pem",
  "csr_path": "AC1/solicitudes/user1.csr.pem"
}
```

Ejemplo de *users.json*

2.3.3 Gestión de Firmas Digitales

En nuestro caso hemos optado por almacenar los datos generados al firmar en los metadatos del archivo cifrado (.json). De esta manera, si y sólo si la verificación de la firma es exitosa se podrá descifrar correctamente el archivo.

```
{
  "filename": "ejemplo.txt",
  "signer": "usuario",
  "enc_key_user": "AHRGpXkS2+opL/JIF5nUMi5vThbuRw51m5peTRXVVhv",
  "enc_key_admin": "XxIJCakG15cWj/mz+zx8PM2f3ka1201xmxEkX+tG3",
  "algorithm": "AES-256-GCM",
  "signature": "e4Cg1Uf2g2yBwbZq5vUW3ZQZyMAK+1FhpMmTujBoyc0Lh"
}
```

Ejemplo de los metadatos de un archivo cifrado

Por tanto, el resultado de la firma digital se refleja en la *signature* o firma, la cual estará vinculada al *signer* o firmante. Incluir el identificador del usuario firmante (*signer*) es crucial a la hora de las verificaciones ya que si no se identifica al usuario emisor de la firma, los administradores no podrán validarla puesto que no poseen la clave pública del firmante.

3. Certificados y PKI (Infraestructura de Clave Pública)

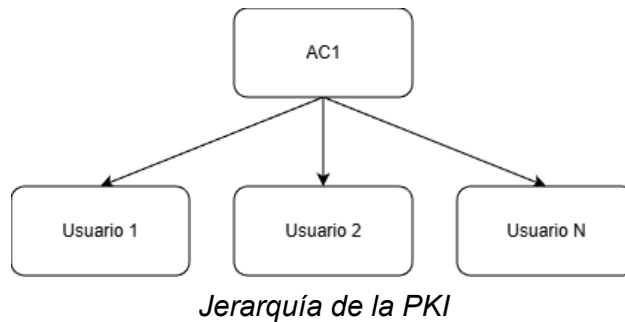
3.1 Jerarquía de autoridades

En una infraestructura de clave pública (PKI), la Autoridad de Certificación (AC) es la entidad encargada de **emitir**, **firmar** y **revocar** certificados digitales. Estos **certificados digitales permiten vincular una clave pública con la identidad del usuario**. Es importante tener claro que los certificados digitales se basan en **cifrado asimétrico**.

En este proyecto hemos implementado una PKI compuesta por una única Autoridad de Certificación raíz (**AC1**) y un conjunto de usuarios finales. Dentro del sistema, la AC1 actúa como la entidad de mayor confianza. Es importante mencionar que su certificado es **autofirmado**, lo que quiere decir que la AC1 misma se certifica su clave pública.

Por tanto, en el sistema **AC1 se encarga** de:

- **Firmar los certificados** generados a partir de las Solicitudes de Firma de Certificados (CSR) de los usuarios.
- Mantener un registro en **index.txt** de los certificados emitidos
- Gestiona la numeración de los certificados mediante el archivo **serial**
- Actúa como autoridad de confianza al **verificar los certificados** de los usuarios en el proceso de descifrado y verificación de firmas digitales.



3.2 Configuración y Justificación

La infraestructura de clave pública o PKI implementada en el proyecto se basa en una configuración sencilla y centralizada. Esta infraestructura está compuesta por los **usuarios finales**, quienes solicitarán los certificados, y por una única **Autoridad de Certificación (AC1)**, que firma las solicitudes de certificado (CSR) de los usuarios y valida los certificados.

Hemos escogido este modelo centralizado y sencillo por varios motivos tanto prácticos como conceptuales. Aunque habría sido posible crear una jerarquía más compleja con autoridades subordinadas, hemos considerado que el modelo que hemos implementado era el más óptimo para cubrir las necesidades de la aplicación. Por tanto, concluimos que implementar una jerarquía de mayor nivel o más profunda no aportaría ventajas al funcionamiento de la aplicación, añadiendo únicamente una complejidad innecesaria.

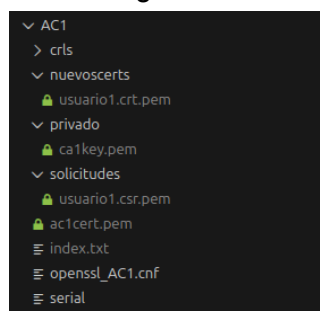
También, esta configuración simplifica en gran medida el uso y la gestión de la PKI ya que: no gestionamos con varias claves privadas de las distintas CA subordinadas, la verificación de los certificados de los usuarios es más ágil, directa y rápida, se minimizan los riesgos de errores.

3.3 Implementación

La Autoridad de Certificación raíz (AC1) utilizada en el proyecto ha sido implementada siguiendo la estructura recomendada en las clases teóricas. Esta organización permite separar adecuadamente las claves privadas, los certificados emitidos, las solicitudes pendientes y los archivos de configuración y control necesarios para el correcto funcionamiento de la autoridad.

3.3.1 Estructura interna de AC1

La estructura de la AC1 se organiza en la siguiente estructura de directorios.



Cada una de las carpetas y archivos cumplen con funciones específicas dentro de la PKI:

- **nuevoscerts/**: directorio cuya función es **almacenar los certificados** de forma organizada.
- **crls/**: carpeta reservada para listas de revocación (CRL). Aunque en este proyecto no se utilizan, OpenSSL la incluye por defecto
- **privado/**: Contiene la **clave privada** de la AC1 (**ac1key.pem**). Esta clave es usada por la autoridad para firmar los certificados X.509. Por tanto, debe permanecer protegida.
- **solicitudes/**: **Almacena las CSR** generadas por los usuarios durante su registro.
- **index.txt**: es una **base de datos** que genera la AC1. En ella podemos encontrar el estado y la información de los certificados emitidos.
- **serial**: Contiene el número de serie que se le otorgará al siguiente certificado emitido.
- **openssl_AC1.cnf**: archivo que **define el comportamiento de la AC1**. Es importante mencionar que sobre este fichero (aportado por el profesor de la asignatura) realizamos dos cambios relevantes para asegurar el correcto funcionamiento de la aplicación.
En primer lugar cambiamos los requerimientos de las solicitudes de certificados ya que en nuestra implementación no requerimos de todos los campos solicitados en el modelo inicial. (Mantuvimos **Common Name** en la solicitud)
En segundo lugar, modificamos los algoritmos usados para hacer los hash en el proceso, siendo el original SHA-1 (roto y vulnerable) y el actualizado **SHA-256** (actual y no vulnerable).
- **ac1cert.pem**: certificado autofirmado de la AC1.

A continuación se adjunta el **comando** utilizado para **crear la AC1**:

El comando se debe ejecutar en el interior del directorio AC1.

```
openssl req -x509 -newkey rsa:2048 -days 360 -out ac1cert.pem -outform PEM -config openssl_AC1.cnf
```

3.3.2 Funcionamiento de la AC1

La AC1 en el sistema cumple con un papel de alta importancia ya que garantiza que todas las operaciones criptográficas se realizan sobre **claves públicas auténticas**.

Las funcionalidades que ejerce la AC1 se pueden dividir en tres fases principales:

- **Registro del usuario**: Cuando un usuario se registra en la aplicación, el sistema **genera** automáticamente una solicitud de firma de certificado o **CSR**, la cual contiene:
 - La **clave pública** del usuario.
 - Su **identidad** (Common Name en la implementación).
 - **Firma** que verifica que el usuario es poseedor de la clave privada.

El CSR en la implementación se genera en la función *generate_csr(username, private_key_pem, password)* y se almacenará en:

```
AC1/solicitudes/ < usuario >.csr.pem
```

- **Firma del certificado**: La firma de la CSR se realiza desde la AC1 mediante **OpenSSL**. El proceso de firma sigue las especificaciones dadas en el fichero openssl_AC1.cnf. El resultado será un **certificado X.509** válido, que se almacenará en:


```
AC1/nuevoscerts/ < serial >.pem
```

Es importante mencionar que tras el usuario iniciar sesión, **el certificado es renombrado**. Las funciones desarrolladas para llevar esta tarea a cabo son: ***get_serial_for_user(username)*** y ***rename_cert(username)***. De tal manera que, cuando el usuario trate de iniciar sesión y la función ***check_cert(username)*** vaya a comprobar si el usuario cuenta con un certificado válido, se llamará a ***rename_cert*** la cual llamará consecuentemente a ***get_serial_for_user***. Por lo tanto, los certificados de los usuarios se almacenan de la siguiente manera:

AC1/nuevoscerts/ < usuario >.crt.pem

También, el **path de los certificados se almacenará en users.json**, lo que permitirá que otros usuarios administradores y el propio sistema puedan cargar el certificado correspondiente para el proceso de verificación.

- **Uso del certificado:** Cada vez que un usuario trate de descifrar un archivo, el sistema hará uso del **certificado digital del usuario firmante** para **validar la firma digital** y garantizar con esto la **autenticidad** de dicho firmante. Dividiremos el proceso en cuatro pasos:

- **Carga del certificado del usuario:** se obtiene el certificado del usuario (ya renombrado) mediante ***load_user_certificate(signer)***
- **Carga del certificado de la AC1:** se obtiene el certificado de la AC1 mediante ***load_ca_certificate()***
- **Validación del certificado digital:** Una vez se tienen los dos certificados, ***cert_user*** y ***cert_ca*** se procede a la **verificación**. En ***verify_signature_signed_by_ca(cert_user, cert_ca)*** se verifica que el **certificado del usuario fue firmado por la AC1**. En caso de que la verificación falle, el archivo no se descifra ya que no se garantiza la identidad del firmante.
- **Verificación de la firma digital del archivo:** Una vez se ha confirmado que el certificado del usuario es válido, el sistema hace uso de su **clave pública para comprobar la firma digital del archivo**. La función que ejecutará esta verificación es ***verify_signature(signature, plaintext, public_key)***. Esta verificación garantiza que el archivo no ha sido modificado, que fue firmado por el usuario correcto. Además con la verificación del certificado anterior, podemos deducir que la clave pública le pertenece a un usuario certificado por la AC1 y por tanto es una firma válida.

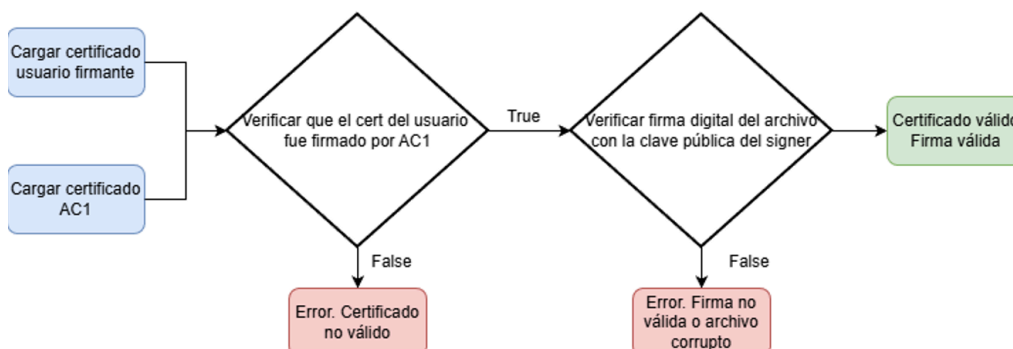


Diagrama de flujo del proceso de verificación del certificado y firma

3.4 Generación de certificados digitales

La aplicación usa un proceso basado en **OpenSSL** para generar los certificados **X.509** de los usuarios.

- **Generación de claves del usuario:** Durante el registro, el sistema genera un par de **claves RSA de 2048 bits** (pública y privada). La privada se cifra con la contraseña del usuario y la pública se utilizará para solicitar el certificado (CSR).
- **Creación de CSR:** la aplicación construye automáticamente una CSR. (Explicación más extendida en 3.3.2)
- **Firma de CSR por AC1:** La AC1 **firma la CSR usando OpenSSL** y su clave privada, *ac1key.pem* (generada al crear la AC1).

`openssl ca -in ./solicitudes/<usuario>.csr.pem -notext -config ./openssl_AC1.cnf`
Este comando se debe ejecutar dentro de la AC1.

Una vez firmado el certificado, OpenSSL actualiza automáticamente *serial* (archivo que hace de contador) e *index.txt* (Base de datos de la AC1). Tras iniciar sesión el usuario, el sistema procede a renombrar el archivo:

`< serial >.pem → < usuario >.crt.pem`

Por último, la ruta del certificado se almacena en *users.json*.

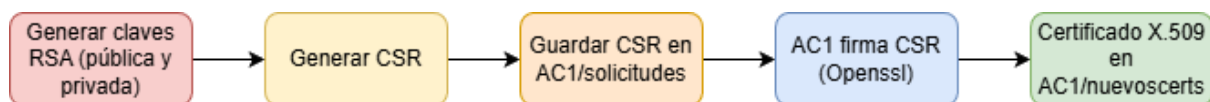


Diagrama de flujo de la emisión de un certificado digital

4. Pruebas realizadas

Para el proyecto se realizaron pruebas de funcionalidad para validar su correcto funcionamiento. Todas las pruebas se han realizado en la interfaz de la aplicación junto a una terminal.

4.1. Registro de usuarios

Caso correcto: usuario válido

- Usuario introducido: user0
- Contraseña: Hola1234!
- Resultado:
 - Se genera salt aleatorio
 - Script deriva el hash correctamente
 - Se generan claves RSA-2048
 - Se crea el CSR
 - Se escribe en users.json

Caso de error: contraseña inválida

- Contraseña que no cumple con la política establecida. Resultado:

“La contraseña no cumple la política: mínimo 8 caracteres, una mayúscula, un número y un símbolo.”

Caso de error: usuario ya existente

- Intento de registrar user0 nuevamente, como resultado salta una ventana:

“El usuario ya existe. Escoja otro nombre.”

4.2. Inicio de sesión

Caso correcto: login exitoso

Tras firmar la CSR del usuario desde la terminal de AC1 se observa que:

- Scrypt valida el hash
- Se descifra la clave privada con la contraseña
- Se localiza el certificado en AC1/nuevoscerts/<usuario>.crt.pem
- Se valida que el certificado está firmado por la CA
- El usuario accede a su almacén seguro

Caso de error: login con contraseña incorrecta

"Usuario o contraseña incorrectos"

Caso de error: login con certificado NO emitido/ CSR no firmada.

Para este caso de error se puede simular si no se ha firmado en la terminal el certificado del usuario. También es posible si borramos/modificamos el certificado en cuestión.

"El certificado no ha sido emitido por la CA"

4.3. Cifrado de archivos

Cuando el usuario introduce un archivo se genera AES-256 aleatorio; nonce de 12 bytes aleatorio; firma RSA-PSS del contenido, y se crea un archivo .bin y otro .json correctamente. En el .json se introduce correctamente los siguientes atributos: signature, signature_algorithm, enc_key_user, enc_key_admin, signer, filename, algorithm.

4.4 Descifrado de archivos

Descifrado por el usuario, donde la aplicación recupera el AES descifrando la clave con RSA. Se verifica la firma digital mediante clave pública del firmante. Se guarda <nombre>_descifrado.ext.

Descifrado por el administrador, como admin, se usa enc_key_admin.

Si manipulamos el archivo .bin se comprueba que no se descifra y salta la ventana con el error: *"Error en descifrado..."*

Caso de error: certificado manipulado

Se modifica el certificado del usuario. Resultado:

"No se pudo descifrar el archivo"

4.5 Verificación de firmas

Caso correcto

La firma digital es correcta. La función *verify_signature* lo confirma

Caso incorrecto

Modificación de la signature en el JSON: *"Firma digital inválida"*

4.6 Verificación de certificados X.509

Caso correcto

El certificado del usuario se valida correctamente. La función *verify_signature_signed_by_ca()* confirma la validez del certificado.

Caso incorrecto: Certificado no emitido por AC1

Intento de login salta error.